

Crypto And C++ Client Threat Model

Date updated: 2026-06-03

Scope

This threat model covers the client-side cryptography and C++ client parts of the Epic Messaging project. It complements the backend threat model in [server/docs/security/backend_threat_model.pdf](#), which covers FastAPI, PostgreSQL, authentication routes, access control, audit logs, deployment headers, rate limiting, and backend blockchain anchor metadata.

The client-side scope is:

- generating and storing device identity keys, signing keys, signed prekeys, and one-time prekeys
- publishing only public key material to the backend
- encrypting outgoing message plaintext before the backend sees it
- decrypting authorized ciphertext after it is received from the backend
- pinning recipient identity keys with a trust-on-first-use model
- storing local tokens, private keys, trust pins, one-time prekey private material, cached messages, and local sender copies
- connecting to the backend over HTTPS with certificate validation
- giving users clear messages when login, TLS, crypto, or verification fails

The backend remains an opaque relay. It may store ciphertext and metadata, but it must not receive plaintext message bodies, private keys, local passphrases, or ratchet/session state.

Rubric Alignment

This document is written to support the CS4455 rubric:

- **Cryptography:** authenticated encryption, key establishment and sender authentication, password/key derivation, design documentation, and interview defence.
- **C++ Programming:** project integration, code structure, functions/classes/OOP, modern C++ and memory safety, documentation, and ability to explain ownership.
- **Computer Networks and Cybersecurity:** HTTPS connectivity, certificate validation, secure storage, input validation, sensitive-data protection, and linkage to the backend threat model.

Assets

Asset	Security property needed	Where handled
Message plaintext	Confidentiality and authenticity	C++/crypto client before backend upload
Message ciphertext	Integrity and tamper detection	AES-256-GCM payload authentication
Sender identity	Authenticity	signed prekey validation and TOFU pins
Recipient identity key	Integrity after first trust	local trust pin store
Device private keys	Confidentiality at rest	encrypted local JSON state
One-time prekey private keys	Confidentiality and single-use semantics	local state and backend public-prekey consumption
Access/refresh tokens	Confidentiality at rest and over network	encrypted local store and HTTPS
Local passphrase	Confidentiality	never sent to backend
Backend certificate	Authenticity and validity	Qt HTTPS certificate validation
Blockchain anchor metadata	Integrity evidence	backend worker, contract, and verification UI

Trust Boundaries

Boundary	Trust decision
User device	Trusted while not compromised; performs E2EE and stores private keys
C++ client local store	Stores sensitive material only after local encryption
Public network	Untrusted; protected by HTTPS/TLS
Backend API	Trusted for authentication and routing, not trusted with plaintext
Backend database	May be read by an honest-but-curious or compromised server
Public key directory	Untrusted on first contact; TOFU reduces later key-substitution risk
Sepolia blockchain	Public immutable evidence only; not a confidentiality mechanism

Attacker Classes

Attacker	Capability
----------	------------

Attacker	Capability
Passive network attacker	Reads traffic between client and public service endpoint
Active network attacker	Modifies, drops, replays, redirects, or injects traffic
Honest-but-curious server	Follows backend code but logs ciphertext, keys, and metadata it can see
Compromised server	Controls API responses, database rows, public key lookup, and message delivery
Malicious authenticated user	Has a valid account and probes other users, keys, messages, and anchors
Stolen laptop attacker	Reads local client files but does not know the local passphrase
Fully compromised client	Reads plaintext and private keys on the endpoint

These attacker classes match the cryptography brief. The backend version of the same model is documented in [server/docs/security/backend_threat_model.pdf](#).

Data Flow

1. User registers or logs in through the C++ client over HTTPS.
2. The client creates or loads local device key material.
3. The client uploads public device keys and public one-time prekeys to the backend.
4. To send a message, the client fetches the recipient's public prekey bundle.
5. The client verifies signed prekey material and checks or creates a TOFU pin.
6. The client encrypts plaintext locally using OpenSSL-backed X25519/HKDF and AES-256-GCM payload encryption.
7. The backend receives only `wire_payload_json`, metadata, and public IDs.
8. The backend stores the encrypted relay record and creates pending blockchain anchor metadata.
9. The recipient client downloads ciphertext, checks local key state, and rejects tampered payloads when AES-GCM authentication fails.
10. The user can ask the C++ client to fetch anchor metadata or use the standalone blockchain verification UI for independent fidelity checks.

STRIDE Threats And Controls

STRIDE	Threat	Client-side control	Backend reference
--------	--------	---------------------	-------------------

STRIDE	Threat	Client-side control	Backend reference
Spoofing	Attacker pretends to be the backend	Real mode requires HTTPS and keeps Qt certificate validation enabled	backend threat model: secure connectivity and headers
Spoofing	Server serves a fake recipient key	signed prekey validation and TOFU pinning detect later key changes	backend threat model: key relay is public-only and authenticated
Tampering	Ciphertext is modified in transit or storage	AES-256-GCM authentication failure rejects modified payloads	backend threat model: backend stores opaque relay payloads
Repudiation	User disputes message origin	sender device identity and signed prekey evidence support origin checks, with audit metadata on backend	backend threat model: audit logging and message metadata
Information disclosure	Server reads message content	plaintext is encrypted before upload; backend sees ciphertext only	backend threat model: no plaintext/private-key columns
Information disclosure	Local state file exposes tokens or private keys	encrypted local JSON store protects sensitive fields	backend threat model: refresh tokens stored as hashes server-side
Denial of service	Server or network drops messages	not fully preventable; user sees delivery/verification errors	backend threat model: rate limits and safe failures
Elevation of privilege	User accesses another user's messages	client cannot rely on UI checks only; backend enforces object-level access	backend threat model: message access-control tests

Security Properties By Attacker

Property	Passive network	Active network	Honest-but-curious server	Compromised server
Message plaintext confidentiality	Holds through TLS and E2EE	Holds if certificate validation succeeds	Holds because server stores ciphertext only	Holds for past ciphertext if client keys remain secret
Ciphertext tamper detection	Not applicable	Holds through TLS and AES-GCM	Holds through AES-GCM	Holds when client decrypts and verifies

Property	Passive network	Active network	Honest-but-curious server	Compromised server
Sender authenticity	Not relevant	Holds after signed prekey/TOFU checks	Holds against normal relay tampering	Weakened on first contact because server can serve fake keys
Metadata privacy	Does not hold	Does not hold	Does not hold	Does not hold
Message delivery availability	Does not hold	Does not hold	Mostly holds if server behaves	Does not hold
Local private-key confidentiality	Not exposed	Not exposed	Not exposed to backend	Holds only if client device and passphrase remain safe

C++ Client-Specific Threats

Threat	Impact	Mitigation or status
Insecure API URL in real mode	Plain HTTP could expose tokens and ciphertext metadata	startup validation rejects non-HTTPS real-mode URLs
TLS certificate validation disabled for demos	MITM could steal credentials and substitute API responses	real mode keeps Qt certificate validation enabled
Mock crypto accidentally used in production	plaintext-shaped demo data could be mistaken for real E2EE	mock mode is separate and real mode requires native crypto
Raw private pointers or manual ownership bugs	crashes, leaks, or unsafe memory behaviour	code uses Qt objects, STL containers, normal values, and interface ownership rather than owning raw pointers
Local state file theft	tokens and private key material exposed	real-mode local store encrypts sensitive fields before writing JSON
Second and later ratchet messages unsupported	follow-up messages may fail to decrypt or lack expected ratchet state	documented limitation; current tests cover first-message X3DH and tamper rejection, not full persisted Double Ratchet state
User cannot interpret failures	demo/interview confusion and unsafe workarounds	presenter reports clear user-facing errors; debug raw errors are opt-in

Crypto-Specific Threats

Threat	Impact	Mitigation or status
--------	--------	----------------------

Threat	Impact	Mitigation or status
Nonce reuse under AES-GCM	breaks confidentiality and integrity for affected key	random 96-bit IVs from CSPRNG; must not reuse key/IV pairs
Weak randomness	predictable keys, IVs, or prekeys	OpenSSL CSPRNG is used in native crypto paths
Server learns shared secret	server could decrypt messages	key agreement happens client-side; private keys are not uploaded
Server swaps public keys on first contact	active MITM for new conversations	TOFU pins identity keys after first trust; users should verify fingerprints where possible
Long-term private key stored unencrypted	stolen laptop exposes future and some past messages	private key material is stored in encrypted local state
Password hash cracking after DB breach	account takeover	backend uses Argon2id; see backend threat model
Replay or duplicated ciphertext	confusing or stale messages	backend IDs and timestamps help detect duplicates; stronger client-side replay tracking remains future work

Known Limitations

- The C++ native crypto currently demonstrates first-message X3DH/AES-GCM encryption and tamper rejection. Persisted Double Ratchet session state is not complete.
- TOFU does not stop a compromised server from substituting keys on first contact. It mainly detects unexpected changes after a key has been pinned.
- The server can always drop messages, delay delivery, hide anchor metadata, or serve malicious API responses.
- Metadata such as sender, recipient, device IDs, timestamps, message sizes, and anchor status remains visible to the backend.
- A fully compromised client device defeats E2EE for that user because the attacker can read plaintext and private keys at the endpoint.
- Blockchain anchoring proves fidelity of recorded digests, not confidentiality and not message delivery.

Interview Defence Points

- TLS protects traffic to the backend; E2EE protects plaintext from the backend. They solve different problems.

- AES-GCM is used because the rubric requires authenticated encryption, not just encryption.
- HKDF provides domain-separated key material from shared secrets.
- Argon2id password hashing belongs to the backend account system; local private key encryption uses a separate local secret and store.
- TOFU is acceptable for the brief but has a clear first-contact MITM limitation.
- The C++ component is meaningful because it is a real client with HTTPS API gateways, command routing, encrypted local state, OpenSSL-backed crypto, and blockchain verification status handling.
- The backend threat model in `server/docs/security/backend_threat_model.pdf` should be read together with this document: backend controls enforce authentication, authorization, validation, audit logging, rate limiting, and database safety, while this document covers client-side E2EE and local storage.

Validation Evidence To Cite

- `client/out/build/mingw-debug/client_tests.exe` passed for parser behaviour, startup validation, HTTPS enforcement, native crypto round trip, AES-GCM tamper rejection, mock crypto isolation, encrypted local-state reload, and blockchain verification status reporting.
- Backend validation evidence is recorded in the backend security PDFs under `server/docs/security/`, especially `backend_threat_model.pdf`, `security_test_results.pdf`, and `security_controls_mapping.pdf`.